

Spécification et preuve de programmes

Solution du devoir 1 : Vérification déductive, mini-langage impératif et logique de Hoare

Alain Giorgetti

Licence d'Informatique de l'Université de Franche-Comté 2024-25

Afin que ce devoir serve aussi d'entraînement aux examens, il est demandé que toutes les réponses soient rédigées **à la main** par l'étudiant.e, sur des feuilles au format A4 portant chacune le nom et le prénom de l'étudiant.e. Ces feuilles doivent être numérisées au format PDF, par scan ou photographie, et déposées sur Moodle dans le devoir prévu à cet effet. Un barème est donné à titre indicatif.

1 Spécifier et vérifier le tri à bulles (5 points)

Cet exercice porte sur le contenu de la page web
https://fr.wikipedia.org/wiki/Tri_%C3%A0_bulles
sur le tri à bulles.

1. Citer une phrase de cette page qui pourrait faire partie de la partie non fonctionnelle d'une spécification fonctionnelle du tri à bulles.

(1 point) La phrase "Pour le tri non optimisé, la complexité en temps est de $\theta(n^2)$, avec n la taille du tableau." spécifie la performance du tri à bulles, ce qui est une partie de sa spécification non fonctionnelle.

2. Citer un extrait d'une page Wikipedia qui spécifie clairement la principale fonctionnalité d'un algorithme de tri.

(1 point) La page https://fr.wikipedia.org/wiki/Algorithme_de_tri dit : "Un algorithme de tri est [...] un algorithme qui permet d'organiser une collection d'objets selon une relation d'ordre déterminée."

3. Préciser si cette spécification est formelle ou informelle, en justifiant votre réponse.

(1 point) Cette spécification est informelle, car exprimée en langue naturelle (ici, en français), pas dans un langage informatique.

4. Citer les extraits de la page Wikipedia du tri à bulles qui décrivent la nature des données d'entrée de cet algorithme.

(1 point) "Le tri à bulles [...] consiste à comparer répétitivement les éléments consécutifs d'un tableau".

5. Si ces extraits ne sont pas assez précis, préciser la nature de ces données d'entrée avec une précision suffisante pour permettre le choix du bon type pour ces données dans une implémentation en langage C.

(1 point) Cet extrait n'est pas précis, car il ne décrit ni la nature des éléments du tableau, ni la nécessité de connaître la taille du tableau. De plus, on doit deviner en lisant le pseudo-code que le premier indice du tableau est 0, et non pas 1. La partie 2.4.3 du cours décrit ces données d'entrée de manière plus précise, avec un premier indice de tableau égal à 1. Pour une implémentation en C, il faudrait préciser quel type d'entiers serait utilisé, par exemple `int`, et réviser toutes les spécifications en supposant que le premier indice du tableau est 0 au lieu de 1.

2 Extrêmes en logique de Hoare (8 points)

On considère un ensemble de nombres muni d'une relation d'ordre totale large (c'est-à-dire réflexive), notée $\leq$. Parmi deux nombres x et y , il y a toujours un plus petit et un plus grand nombre selon cet ordre.

- Donner un triplet de Hoare sans précondition dont le programme calcule dans une variable s le plus petit des deux nombres x et y et dans une variable l le plus grand de ces deux nombres. La postcondition de ce triplet doit déclarer que s est plus petit que l est que ces deux nombres se partagent les nombres x et y , avec un "ou".

(2 points)

$\{\} \text{ if } x \leq y \text{ then } s := x; l := y \text{ else } s := y; l := x \text{ fi } \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$

- On considère la variante de la logique de Hoare du cours dans laquelle la règle (*consequence*) est remplacée par les deux règles suivantes :

$$\frac{P \Rightarrow P' \quad \{P'\} c \{Q\}}{\{P\} c \{Q\}} \text{ (precondition)} \quad \frac{\{P\} c \{Q'\} \quad Q' \Rightarrow Q}{\{P\} c \{Q\}} \text{ (postcondition)}$$

En appliquant une par une les règles de cette logique de Hoare, démontrer formellement **dans un tableau** que ce triplet de Hoare est valide.

(4 points : 0,5 par ligne sauf **1**, **8** et **10**)

Dans la logique de Hoare, une preuve de validité de ce triplet de Hoare est :

No	Triplet ou formule	Justification
1	$\{\} \text{ if } x \leq y \text{ then } s := x; l := y \text{ else } s := y; l := x \text{ fi } \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$	(if-then-else) 2 3
2	$\{x \leq y\} s := x; l := y \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$	(seq.) 4 5
3	$\{\neg(x \leq y)\} s := y; l := x \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$	(seq.) 6 7
4	$\{x \leq y\} s := x \{s \leq y \wedge (s = x \wedge y = y \vee s = y \wedge y = x)\}$	(precond.) 8 9
5	$\{s \leq y \wedge (s = x \wedge y = y \vee s = y \wedge y = x)\} l := y \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$	(assign.)
6	$\{\neg(x \leq y)\} s := y \{s \leq x \wedge (s = x \wedge x = y \vee s = y \wedge x = x)\}$	(precond.) 10 11
7	$\{s \leq x \wedge (s = x \wedge x = y \vee s = y \wedge x = x)\} l := x \{s \leq l \wedge (s = x \wedge l = y \vee s = y \wedge l = x)\}$	(assign.)
8	$x \leq y \Rightarrow x \leq y \wedge (x = x \wedge y = y \vee s = y \wedge y = x)$	(voir (1))
9	$\{x \leq y \wedge (x = x \wedge y = y \vee s = y \wedge y = x)\} s := x \{s \leq y \wedge (s = x \wedge y = y \vee s = y \wedge y = x)\}$	(assign.)
10	$\neg(x \leq y) \Rightarrow y \leq x \wedge (y = x \wedge x = y \vee y = y \wedge x = x)$	(voir (2))
11	$\{y \leq x \wedge (y = x \wedge x = y \vee y = y \wedge x = x)\} s := y \{s \leq x \wedge (s = x \wedge x = y \vee s = y \wedge x = x)\}$	(assign.)

(1 point pour chaque justification)

(1) L'implication $x \leq y \Rightarrow x \leq y \wedge (x = x \wedge y = y \vee s = y \wedge y = x)$ est évidente car elle est de la forme $A \Rightarrow A \wedge (\text{vrai} \vee \dots)$.

(2) L'implication $\neg(x \leq y) \Rightarrow y \leq x \wedge (y = x \wedge x = y \vee y = y \wedge x = x)$ est successivement équivalente à

$$\neg(x \leq y) \Rightarrow y \leq x \wedge (y = x \wedge x = y \vee \text{vrai})$$

$$\neg(x \leq y) \Rightarrow y \leq x \wedge \text{vrai}$$

$$\neg(x \leq y) \Rightarrow y \leq x$$

qui est évidente car $\neg(x \leq y) \Leftrightarrow y < x$.

3 Présentation par annotations principales (7 points)

En pratiquant la rédaction de preuves de triplets de Hoare sous la forme d'un tableau ou d'arbres de preuve, on constate qu'on doit souvent écrire plusieurs fois les mêmes formules ou portions de code. Cet exercice propose une présentation plus légère d'une telle preuve, dite "par annotations principales". Elle consiste à écrire le triplet de Hoare avec une instruction par ligne et indentation, comme dans la figure 1, et à intercaler des annotations de la forme $\{\dots\}$ entre certaines instructions. Ces annotations sont soulignées dans la figure 1. Chaque annotation a_i doit être une formule vraie à l'endroit du programme où elle est écrite.

```

{ $n \geq 0$ }
i := 0;
{ $a_1$ }
f := 1;
{ $a_2$ }
while i < n do
  { $a_3$ }
  i := i + 1;
  { $a_4$ }
  f := f * i
  { $a_5$ }
od
{ $a_6$ }
{f = n!}

```

FIGURE 1 – Exemple de programme annoté

Même si on peut ajouter une telle annotation avant et après chaque instruction, comme c'est le cas dans la figure 1, on peut aussi alléger la présentation en n'écrivant que certaines d'entre elles, dites "principales". Par exemple, on peut omettre les annotations $\{a_1\}$ et $\{a_4\}$ de la figure 1, car elles sont situées entre deux affectations.

Par ailleurs, on peut déduire des règles (*consequence*) et (*while*) la forme générale des quatre annotations principales d’une boucle **while**, présentée dans la figure 2, où I est un invariant de boucle.

$$\begin{array}{l} \{I\} \\ \text{while } b \text{ do} \\ \quad \{b \wedge I\} \\ \quad c \\ \quad \{I\} \\ \text{od} \\ \{\neg b \wedge I\} \end{array}$$

FIGURE 2 – Forme générale des annotations d’une boucle **while**

Par conséquent, une preuve par annotations principales du triplet de Hoare

$$\{n \geq 0\} \ i := 0; \ f := 1; \ \text{while } i < n \ \text{do } i := i + 1; \ f := f * i \ \text{od } \{f = n!\}$$

peut commencer par le contenu de la figure 3, où les quatre annotations de la boucle respectent la forme générale de la figure 2, en choisissant $i \leq n \wedge f = i!$ pour l’invariant I .

$$\begin{array}{l} \{n \geq 0\} \\ i := 0; \\ f := 1; \\ \{i \leq n \wedge f = i!\} \\ \text{while } i < n \ \text{do} \\ \quad \{i < n \wedge i \leq n \wedge f = i!\} \\ \quad i := i + 1; \\ \quad f := f * i \\ \quad \{i \leq n \wedge f = i!\} \\ \text{od} \\ \{i \not\leq n \wedge i \leq n \wedge f = i!\} \\ \{f = n!\} \end{array}$$

FIGURE 3 – Annotations principales du triplet de Hoare

Dans ce code annoté, on “voit” qu’il reste à démontrer deux triplets de Hoare et une implication :

- Le premier triplet va de la précondition jusqu’au début de la boucle, c’est

$$\{n \geq 0\} \ i := 0; \ f := 1 \ \{i \leq n \wedge f = i!\}.$$

- Le second triplet formalise que le corps c de la boucle préserve l’invariant I , c’est

$$\{i < n \wedge i \leq n \wedge f = i!\} \ i := i + 1; \ f := f * i \ \{i \leq n \wedge f = i!\}.$$

- L’implication restante

$$i \not\leq n \wedge i \leq n \wedge f = i! \Rightarrow f = n!$$

formalise que la conjonction de la négation de la condition de boucle b et de l’invariant I impliquent la postcondition du triplet initial.

1. Prouver séparément ces deux triplets et cette implication.

(3 points) Voir cours.

2. Rédiger une présentation par annotations principales de la preuve du triplet de Hoare

$$\{n \geq 0\} \ r := 0; \text{while } (r+1)^2 \leq n \text{ do } r := r+1 \text{ od } \{r^2 \leq n < (r+1)^2\}$$

de l'exercice 3 de la feuille d'exercices sur la logique de Hoare.

(1 point)

```

{ n ≥ 0 }
r := 0;
{ r² ≤ n }
while (r + 1)² ≤ n do
  { (r + 1)² ≤ n ∧ r² ≤ n }
  r := r + 1
  { r² ≤ n }
od
{ (r + 1)² ≤ n ∧ r² ≤ n }
{ r² ≤ n < (r + 1)² }

```

Il reste à démontrer deux triplets de Hoare et une implication :

- Le premier triplet va de la précondition jusqu'au début de la boucle, c'est

$$\{n \geq 0\} \ r := 0 \ \{r^2 \leq n\}.$$

- Le second triplet formalise que le corps c de la boucle préserve l'invariant I , c'est

$$\{(r+1)^2 \leq n \wedge r^2 \leq n\} \ r := r+1 \ \{r^2 \leq n\}.$$

- L'implication restante

$$(r+1)^2 \leq n \wedge r^2 \leq n \Rightarrow r^2 \leq n < (r+1)^2$$

formalise que la conjonction de la négation de la condition de boucle et de l'invariant impliquent la postcondition du triplet initial.

(1 point) Une preuve par arbre du premier triplet est

$$\frac{n \geq 0 \Rightarrow 0^2 \leq n \quad \overline{\{0^2 \leq n\} \ r := 0 \ \{r^2 \leq n\}} \text{ (assignment)}}{\{n \geq 0\} \ r := 0 \ \{r^2 \leq n\}} \text{ (precondition)}$$

dont la première prémisse est évidemment vraie.

(1 point) Une preuve par tableau du second triplet est

N°	Triplet ou formule	Justification
1	$(r+1)^2 \leq n \wedge r^2 \leq n \Rightarrow (r+1)^2 \leq n$	OK
2	$\{(r+1)^2 \leq n\} \ r := r+1 \ \{r^2 \leq n\}$	(assignment)
3	$\{(r+1)^2 \leq n \wedge r^2 \leq n\} \ r := r+1 \ \{r^2 \leq n\}$	(precondition)

(1 point) L'implication restante $(r+1)^2 \leq n \wedge r^2 \leq n \Rightarrow r^2 \leq n < (r+1)^2$ est évidente, puisque $(r+1)^2 \leq n$ est équivalent à $(r+1)^2 > n$ et donc à $n < (r+1)^2$.